

Email Spam Classifier - Post Project Report

Machine Learning-Based Text Classification System

Author: Vidhya Vinothkumar (Team 3)

Project Type: Individual Project

EXECUTIVE SUMMARY

This project presents a machine learning system that classifies messages as spam or legitimate (ham). It is implemented as an end-to-end pipeline that processes raw text, converts it into numerical features, trains multiple models, and produces predictions with explanations.

Spam messages are common and can be misleading. Automating their detection improves safety and reduces manual effort. The system uses Python, Natural Language Processing (NLP), TF-IDF feature extraction, and supervised machine learning.

Beyond prediction, the system provides probability scores, keyword-based explanations, and risk levels, making outputs interpretable and practical.

DATASET DESCRIPTION

The system uses the SMS Spam Collection dataset, downloaded using KaggleHub.

The dataset contains labeled messages (spam/ham). The distribution is imbalanced, with more ham than spam, which reflects real-world conditions and can bias models toward the majority class.

Duplicate messages are removed to prevent repeated patterns from biasing the model and to improve generalization.

SYSTEM WORKFLOW

Text → Cleaning → TF-IDF → Model → Prediction → Explanation → Risk

This pipeline ensures consistent processing for both training and prediction.

TECHNICAL IMPLEMENTATION

1. **Data Loading** The dataset is downloaded using KaggleHub and loaded with Pandas for processing.

2. **Text Cleaning** Preprocessing includes:

- Converting text to lowercase
- Removing non-alphabet characters using regular expressions
- Removing stopwords using NLTK

This reduces noise and keeps meaningful words.

3. **Feature Engineering (TF-IDF)** TF-IDF is applied with:

- **max features** = 3000 (limits vocabulary to reduce overfitting and improve speed)
- **ngram range** = (1,2) (captures unigrams and bigrams like “free offer”)

This converts text into numerical vectors that models can use.

4. **Label Encoding** Labels (spam/ham) are converted to numeric values using LabelEncoder (spam=1, ham=0), since models require numerical inputs.

5. **Train-Test Split** Data is split into training and testing sets (80:20). This allows evaluation on unseen data and helps prevent overfitting.

6. **Model Training (Loop-Based)** Multiple models are trained in a loop:

- Naive Bayes
- Logistic Regression
- Support Vector Machine (SVM)

A uniform pipeline ensures fair comparison.

7. **Evaluation and Results Storage** Performance is computed using Accuracy, Precision, Recall, and F1-score. Results are stored in a structured dictionary, enabling automatic comparison across models.

8. **Metric Interpretation**

- **Precision:** proportion of predicted spam that is truly spam (critical to avoid false positives)
- **Recall:** proportion of actual spam correctly detected
- **F1-score:** balance between precision and recall

9. **Model Selection (Precision-Based)** The best model is selected based on the highest precision to minimize important messages being misclassified as spam.

10. **Global Spam Analysis** Average TF-IDF scores across spam messages are computed to identify globally important spam-indicating words. This provides dataset-level insights beyond individual predictions.

11. **Prediction System** Incoming messages are processed through the same pipeline (cleaning → vectorization → model) to ensure consistency.

12. **Probability Extraction** Prediction probability is obtained using `predict_proba`. The value `[0][1]` represents the probability of the spam class in binary classification.
13. **Keyword Extraction** Top keywords are selected by sorting TF-IDF scores and choosing the highest contributing features for the given message.
14. **Risk Classification** Probability is mapped to risk levels:
 - High Risk ($\geq 90\%$)
 - Medium Risk (70–90%)
 - Low Risk ($< 70\%$)These thresholds provide intuitive decision confidence similar to real filtering systems.
15. **Explanation System** A rule-based layer generates explanations based on probability levels, combining machine learning predictions with interpretable logic.
16. **Error Handling** Empty input is detected and handled to prevent invalid predictions.

MODEL COMPARISON

- **Naive Bayes** – Fast and highly effective for text data
- **Logistic Regression** – Stable and consistent performance
- **SVM** – Captures complex decision boundaries

Naive Bayes achieved the highest precision and was selected as the final model.

FEATURE ENGINEERING

TF-IDF assigns importance to words based on frequency and uniqueness. Words like “free”, “win”, and “offer” receive higher weights, while common words receive lower importance. The inclusion of bigrams improves context understanding and enhances classification performance.

SOURCE CODE AND OUTPUT

EMAIL SPAM CLASSIFIER

IMPORT

```
import pandas as pd
import numpy as np
import re
import nltk
import os
import kagglehub

nltk.download('stopwords')

from nltk.corpus import stopwords
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

import matplotlib.pyplot as plt
import seaborn as sns
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

#LOAD DATA

```
#Load Data
path = kagglehub.dataset_download("uciml/sms-spam-collection-dataset")
print("Dataset Path:", path)

print("Files:", os.listdir(path))

file_path = os.path.join(path, "spam.csv")
df = pd.read_csv(file_path, encoding='latin-1')

# Clean columns

df = df[['v1', 'v2']]
df.columns = ['label', 'text']

# Remove duplicates

df.drop_duplicates(inplace=True)

print("\n📊 Dataset Info:")
print(df['label'].value_counts())
```

```
Downloading to /root/.cache/kagglehub/datasets/uciml/sms-spam-collection-dataset/1.archive...
100%|██████████| 211k/211k [00:00<00:00, 28.7MB/s]Extracting files...
Dataset Path: /root/.cache/kagglehub/datasets/uciml/sms-spam-collection-dataset/versions/1
Files: ['spam.csv']
```

```
📊 Dataset Info:
label
ham      4516
spam      653
Name: count, dtype: int64
```

TEXT CLEANING (NLP)

```
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z', ' ', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

df['cleaned'] = df['text'].apply(clean_text)

# Demo cleaning

sample = "Win ₹10000 NOW!!! Click here!!!"
print("\n🔍 Cleaning Demo:")
print("Original:", sample)
print("Cleaned :", clean_text(sample))
```

🔍 Cleaning Demo:
Original: Win ₹10000 NOW!!! Click here!!!
Cleaned : win click

FEATURE ENGINEERING

```
vectorizer = TfidfVectorizer(max_features=3000, ngram_range=(1,2))
X = vectorizer.fit_transform(df['cleaned'])

encoder = LabelEncoder()
y = encoder.fit_transform(df['label']) # spam=1
```

SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

TRAIN MODELS

```
models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "SVM": SVC(probability=True)
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    results[name] = {
        "model": model,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred)
    }
```

SHOW RESULTS

```
print("\nMODEL COMPARISON:\n")

for name, res in results.items():
    print(f"{name}:")
    print(f" Accuracy : {res['accuracy']:.4f}")
    print(f" Precision: {res['precision']:.4f}")
    print(f" Recall   : {res['recall']:.4f}")
    print(f" F1 Score  : {res['f1']:.4f}")
    print("-" * 30)

# Best model (based on precision)

best_model_name = max(results, key=lambda x: results[x]['precision'])
best_model = results[best_model_name]['model']

print(f"\nBest Model: {best_model_name}")

print("\nInsight: Precision is prioritized to avoid misclassifying important emails as spam.")

***
MODEL COMPARISON:

Naive Bayes:
Accuracy : 0.9816
Precision: 1.0000
Recall   : 0.8571
F1 Score : 0.9231
-----
Logistic Regression:
Accuracy : 0.9681
Precision: 0.9808
Recall   : 0.7669
F1 Score : 0.8608
-----
SVM:
Accuracy : 0.9797
Precision: 0.9912
Recall   : 0.8496
F1 Score : 0.9150
-----
```

SPAM WORDS

```
feature_names = vectorizer.get_feature_names_out()
spam_indices = np.where(y == 1)[0]

spam_matrix = X[spam_indices].toarray()
avg_scores = np.mean(spam_matrix, axis=0)

top_indices = avg_scores.argsort()[-10:][::-1]

print("\n🔥 Top Global Spam Words:")
for i in top_indices:
    print(feature_names[i])
```

```
🔥 Top Global Spam Words:
call
free
txt
text
stop
mobile
claim
reply
prize
ur
```

PREDICTION SYSTEM

```
def get_keywords(text):
    cleaned = clean_text(text)
    vector = vectorizer.transform([cleaned])

    scores = vector.toarray()[0]
    feature_names = vectorizer.get_feature_names_out()

    top_indices = scores.argsort()[-5:][::-1]
    keywords = [(feature_names[i], round(scores[i], 3)) for i in top_indices if scores[i] > 0]

    return keywords

def get_explanation(prob):
    if prob > 0.9:
        return "Strong spam indicators with high-risk promotional language."
    elif prob > 0.7:
        return "Moderate spam signals detected."
    else:
        return "Low spam probability, likely safe message."

def get_risk(prob):
    if prob > 0.9:
        return "HIGH 🚨"
    elif prob > 0.7:
        return "MEDIUM ⚠️"
    else:
        return "LOW ✅"

def predict_email(text):
    if text.strip() == "":
        print("⚠️ Please enter a valid message")
        return
```

```
def predict_email(text):
    if text.strip() == "":
        print("⚠️ Please enter a valid message")
        return

    cleaned = clean_text(text)
    vector = vectorizer.transform([cleaned])

    pred = best_model.predict(vector)[0]
    prob = best_model.predict_proba(vector)[0][1]

    keywords = get_keywords(text)
    explanation = get_explanation(prob)
    risk = get_risk(prob)

    print("\n=====")
    print(" Message:", text)
    print(" Prediction:", "SPAM 🚨" if pred == 1 else "HAM ✅")
    print(f" Probability: {prob*100:.2f}%")
    print(f" Risk Level: {risk}")

    print("\n Top Keywords:")
    for word, score in keywords:
        print(f" - {word} ({score})")

    print("\n Explanation:")
    print(explanation)
    print("=====")
```

TEST

```
predict_email("Congratulations! You won a FREE iPhone. Click now!")
predict_email("Hey, are we meeting at 5 PM today?")
```

=====

 Message: Congratulations! You won a FREE iPhone. Click now!

 Prediction: SPAM 

 Probability: 54.71%

 Risk Level: LOW 

 Top Keywords:

- click (0.684)
- congratulations (0.622)
- free (0.38)

 Explanation:

Low spam probability, likely safe message.

=====

=====

 Message: Hey, are we meeting at 5 PM today?

 Prediction: HAM 

 Probability: 1.38%

 Risk Level: LOW 

 Top Keywords:

- meeting (0.558)
- pm (0.54)
- hey (0.458)
- today (0.433)

 Explanation:

Low spam probability, likely safe message.

=====

ADVANTAGES

- Automated and fast classification
- High precision with reduced false positives
- Explainable predictions (keywords and probability)
- Dataset-level insights via global analysis
- Efficient and reusable pipeline

LIMITATIONS

- Limited to text-based analysis
- Performance depends on dataset quality
- Cannot detect advanced phishing techniques

FUTURE SCOPE

- Integration with email platforms
- Real-time deployment
- Enhanced phishing detection
- Extension using deep learning models

CONCLUSION

The Email Spam Classifier demonstrates the effective use of NLP and machine learning for spam detection. The system combines accurate prediction with explainability and decision logic, making it suitable for real-world applications.